



【wasmtime-wasi】在 Rust 中执行 Wasi 协议的 wasm



锦恢

<https://kirigaya.cn>[关注他](#)

8 人赞同了该文章 >

[收起](#)

本文同步于我的个人博客：

协议与生态共融

汇尘轩 - 锦恢的博客

kirigaya.cn/blog/article?seq=286

Wasi 1.0 协议与生态共融

目前，py，js，java，rust，一堆语言都有人开发了 wasi 协议的支持。这意味着，你使用任何一种语言编译成 wasi wasm 后，就可以在任意一个其他语言的里面去执行这个 wasm。至此，wasm 已经和 web 没啥太大的关系了。这个 wasm 还具备 IO 权限，你可以在编写这个 wasm 时读入文件或者读出文件，这意味着 wasi 协议的 wasm 不再会在 IO 上发生显著的性能损失了。



这意味着所有底层基础库的语言生态可以共用了，这种共用基于的这个 IR⁺（也就是 wasm）还是跨平台的，这难道不是非常激动人心吗？当然，wasm 至此真的和 web 没啥特别大的关系了，web 之于 wasm 只是可选项了，简直倒反天罡，但是优秀的表演总是需要出乎意料的事情出现，不是吗？哈哈哈？

rust 和 nodejs 对 wasi 的支持都非常好，事实上，网上目前最热门的两个 wasi 框架都是 rust 写的，所以，本文演示一下如何在 rust 中去执行一个 wasm。

安装

首先，创建一个新项目，并加入必要的库：

```
cargo+ new yosys && cd yosys
cargo add anyhow
cargo add wasmtime
cargo add wasmtime-wasi
```

我使用的版本如下：

```
[dependencies]
anyhow = "1.0.94"
wasmtime = "28.0.0"
wasmtime-wasi = "28.0.0"
```

wasmtime-wasi 默认的 API 只支持 component 类似导出的 wasm，这个留到以后写。我现在的 wasi wasm 是基于 module 的。在之前的文章中，我们可以在 nodejs 环境中轻松使用编译出来的 wasm，因为 nodejs 原生就是支持 wasi wasm 的：

```
'use strict';
const fs = require('fs');
const { WASI } = require('wasi');
const path = require('path');

const wasi = new WASI({
  version: 'preview1',
  args: [
    'yosys',
    '-s',
    '/home/dide/project/yosys/half_adder.ys'
  ],
  env: process.env,
  preopens: {
    '/': '/',
    // 自定义映射，./ 必须是前面两个文件所在的路径
    '/ws': './',
    // share 就是上文提到的编译得到的 share 文件夹
    '/share': '/path/to/share'
  },
  stdin: process.stdin.fd, // 绑定到 Node.js 的标准输入
  stdout: process.stdout.fd, // 绑定到 Node.js 的标准输出
  stderr: process.stderr.fd, // 绑定到 Node.js 的标准错误
});

(async () => {
  const wasmPath = path.join(__dirname, 'yosys.wasm');
  const binary = await fs.readFileSync(wasmPath);

  const wasm = await WebAssembly.compile(binary);
```

// 启动 WASI 实例

```
const exitCode = wasi.start(instance);

console.log('执行结束, exit code:', exitCode);
})();
```

为了方便观众理解，我们下面就把上面的过程，用 rust 转写一遍：

```
use std::path::Path;

use anyhow::{Ok, Result};
use wasmtime_wasi::preview1;

use wasmtime_wasi::{DirPerms, FilePerms, WasiCtxBuilder};
use wasmtime::*;

struct MyState {
    wasi: preview1::WasiP1Ctx,
}

impl MyState {
    fn new() -> Result<MyState> {

        let wasi = WasiCtxBuilder::new()
            .args(&[
                "yosys",
                "-s",
                "/ws/half_adder.js"
            ])

            // wasi 内访问 /share 开头的都会在 runtime 中以 ./static/share 替换
            .preopened_dir(
                Path::new("./static/device"),
                "/share",
                DirPerms::all(),
                FilePerms::all()
            )?

            // wasi 内访问 /ws 开头的都会在 runtime 中以 ./test 替换
            .preopened_dir(
                Path::new("./test"),
                "/ws",
                DirPerms::all(),
                FilePerms::all()
            )?

            .inherit_stdio()
            .build_p1();

        Ok(MyState {
            wasi
        })
    }
}

fn main() -> Result<()> {
    let engine = Engine::default();
    let module = Module::from_file(&engine, "./static/yosys.wasm")?;

    // 配置 WASI 上下文
    let my_state = MyState::new()?;
    let mut store = Store::new(&engine, my_state);
    let mut linker = Linker::new(&engine);
```

```
// 实例化模块
let instance = linker.instantiate(&mut store, &module)?;
let start = instance.get_typed_func::<(), ()>(&mut store, "_start"?);

match start.call(&mut store, ()) {
    std::result::Result::Ok(_) => {
        println!("执行成功");
        Ok(())
    }
    std::result::Result::Err(_) => {
        Ok(())
    }
}
```

编译运行：

```
cargo build --release
.\target\release\yosys.exe
```

可以成功执行并生成综合之后的电路。

当然，对于其他的 wasm 负载，比如说 ffmpeg 呀什么的，如果你看了一个用 C 语言实现的库，但是苦于没有同等级的 rust 实现的库，那么可以考虑用 wasi sdk 把这个 C 库编译成 wasm，然后再用上文的方法在你的 rust 应用中去使用它。

这是多么让人感觉舒畅的事情！不多说了，我要去继续开发布线算法了。



编辑于 2024-12-21 19:42 · IP 属地江苏

内容所属专栏



K的Web全栈
搭建网站，是为了更好地展示AI

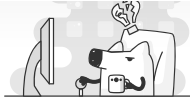
订阅专栏

WASM rust

▲ 赞同 8 ▼ ● 添加评论 ↗ 分享 ♥ 喜欢 ★ 收藏 📄 申请转载 ...



理性发言，友善互动



推荐阅读



给 Web 前端工程师看的用 Rust 开发 wasm 组件实战 | 京东云...

京东云

教你把go rust wasm结合在一起装逼

wasm将火如果你还没意识到，那抓紧学习起来了，正如docker创始人的说法，wasm早出现几年就没docker啥事了。那即使现在出来也不晚。Polkadot的核心runtime也用wasm跑的，包括区块链领域的...
中本呆



Rust + Wasm：2MB应用，实现超轻量AI推理

WasmEdge

rust-wasm + react 新习

本文未经许可，禁止任何形载!!!发现必究!!!所第一时间发布在公众号：超吉姆。这篇文章花了很多心望能够帮助到大家，同时也家能够点赞、关注，这是我

超级英雄吉... 发表于超级